

P?S/8 PAL Command-line Option Switches [and related subjects].

Copyright © 1975-1989 Charles Lasner Associates

Copyright © 1990-2015 »CLA Systems

CLASystems@gmail.com

Last edit: 23-Aug-2015 cjl

P?S/8 PAL Version 8S is a general-purpose PDP-8 assembler for PDP-8 Assembly Language [PAL]. It is distributed as part of P?S/8 Version 8Z. Unlike competing assemblers [TOPS-10 PAL10 and OS/8 PAL8], there are many command-line option switches; this is necessary to allow the flexible and modular sections of P?S/8 PAL to be invoked as needed.

The command-line options are passed in the same manner as OS/8 CCL commands. [Note: P?S/8 supports several command prompt variations. The one used throughout this document is identical to TOPS-10 and OS/8 to aid in various comparisons. <SP> and <HT> are freely allowed between command elements to enhance command readability and are not otherwise required.]

Typical command examples [in slightly simplified form] would be formed as follows:

.PAL output files < input files option switches

or

.PAL input files > output files option switches

Note: P?S/8 allows command formation in either order as shown above; this allows the user to work in a familiar environment; the commands are generally similar to the various concise command implementations found in other DEC operating systems.

The option switches can be placed anywhere in the command line after the program name [PAL].

For more information about P?S/8 command-line variations, consult the [P?S/8 Keyboard Monitor Command Guide](#) available as a separate document.

PAL Input Files.

The program input text passed to PAL is generally the textual contents of a series of Tiny File System [TFS] files concatenated together in the command. Each TFS file is exactly 2048 12-bit words long; in general, each file will tend to contain roughly 3/4 of this capacity as six-bit bytes in the P?S/8 six-bit ASCII text character set. This roughly translates to about 3000 characters per TFS file.

The total number of files that can be passed to any P?S/8 system program is 17. Since several file positions may be needed for output files, the actual number of input files tends to be slightly less.

Note: P?S/8 also supports extended-length input files, each of which can contain considerably more characters than the entirety of 17 TFS files. For more information, consult documentation of the P?S/8 OS8CON program or other similar file conversion programs available as separate documents.

PAL Output Files.

In P?S/8, all TFS files are fixed size of 2048 12-bit words. TFS binary files are created in the famous *Slurp* format which is far more efficient than files based on paper-tape frame images. A typical TFS binary file contains nearly 6/7 as much as a block-format core-image file yet no block structure is used or required. Binary loaders for TFS binary files are available on all P?S/8 configurations. A generic virtual loader is always available; most systems also support a device-specific binary file loader [which will be the default loader if present]. Command line switches control which loader is used in specific situations initiated by P?S/8 PAL [assemble, load and go *or* assemble, map, load and go].

At the end of the assembly, information is displayed regarding various assembly particulars; the count of binary output files actually created is given [allowing any excess passed output files to be used for other purposes]. If insufficient files are passed, a binary overflow [BO] error will be displayed which aborts the assembly.

Command-line Option Switches.

The command-line option switches are passed in a manner consistent with OS/8 CCL. Two variant forms are supported:

1) Individual option switches are preceded by / such as:

/M /N /L

2) Grouped switches are preceded by (such as:

(MNL)

Note: The trailing) is optional unless needed to delimit the option group from another command section.

Both forms of option switches can be used together; option switches may be given redundantly without error.

Passed Numeric Value.

P?S/8 PAL can chain to P?S/8 BIN to load the binary files created by the assembly. If desired, the binary program can be automatically started at an address expressed as =xxxx where xxxx is up to four octal digits in the range of 0 through 7776; for programs starting in extended memory, the /1 through /7 switches provide the starting field.

while the specific value =7777 is not permitted, the absence of the passed value causes P?S/8 BIN to use an internal safe halt location. [The exact value depends on other loading options.] [Internally, P?S/8 passes =7777 to system programs when the explicit numerical value is not given on the command line.]

Note: Assembly errors abort the chaining process to P?S/8 BIN.

Command-line option switches grouped by general function.

1. Output File Options.

If explicit binary output files are not given, the following option switches can be used:

/B The % file on the P?S/8 system device bootup logical unit [see description of the /U option switch below] is used as the [first] binary output file.

/D If the /B option switch is set, the \$ on the P?S/8 system device bootup logical unit [see description of the /U option switch below] is used as the second binary output file.

Note: If the second binary output file is not required, the /D option switch will be ignored; assembly statistics will indicate the actual output file count.

/U Change the logical unit that applies with the /B and /D option switches set as described above from the system device bootup logical unit to the result of the operation [system device bootup logical unit XOR 1].

The use of the % and \$ files can greatly speed up various system operations due to their fixed placement near the physical beginning of the device. Additionally, they are not subject to entries within the TFS directory; as such, certain operations have significantly lower overhead when these files are used strategically. On tape-based systems, throughput can be further improved by using a logical unit different from the system bootup logical unit.

Note: The /B, /D and /U option switches are provided to simplify the process of specifying output files. Since all file specifications can have optional device units, any combination of these files on any logical unit can be explicitly stated; the option switches provide the most commonly used output file combinations in a succinct form.

If more than two binary output files are required, or the created binary files should be written to permanent TFS file slots, the command line given must use an explicit list of output files.

2. Chaining Options.

If desired, automatic chaining options can be invoked to further process the output files created during the assembly:

/G The output files created during the assembly are passed as input files to the P?S/8 BIN utility for loading [or punching binary paper-tape as described below]. Chaining to P?S/8 BIN is prevented if there are any assembly errors. Additional loading-specific option switches may be used [see the description of the /V, /H, /Z and /I option switches and the =xxxx passed numerical value to set the starting address in conjunction with the /1 through /7 option switches to set the starting memory field in Section 9 of this document].

/M The output files created during the assembly are passed as input files to the P?S/8 MAP [bitmap] utility to create a formatted bitmap of the assembled program.

Note: The format of the bitmap output is generally consistent with the output of PAL10 after any FIELD directive [or end of assembly] and is a superset of OS/8 BITMAP. The title format and pagination details are the same as those established for the assembler listing output [if any] and statistics pages.

When P?S/8 MAP is used without chaining, there are options to limit which memory fields are subject to mapping; during the chaining process, these options are not allowed; as such, all memory fields are eligible for bitmap output.

It is possible to use both chaining options in a single command; the chain to P?S/8 MAP will always be performed first; the further chain to P?S/8 BIN will be aborted if any assembly errors occurred.

3. Pass Control Options.

/K Perform a two-pass assembly; the default is a single assembly pass.

Unlike OS/8 PAL8 which always performs two [or three] passes, P?S/8 PAL can perform a useful single-pass assembly [which is consistent with the paper-tape system assembler PAL III].

At the end of pass one, all symbols still not defined are shown on the system console in sorted order; the value indicated is the address within the assembly where each undefined symbol was first encountered. This facilitates early development of the program when it is likely that portions of the program have not yet been written, variables and temporaries are not yet defined [or perhaps are the object of typographical errors which can cause problems when certain symbols have names similar to other symbols] or some other problem exists that can be resolved merely by reference to the list of undefined symbols.

Contrast this with OS/8 PAL8 which does not support a mechanism to merely determine undefined symbols. The only facility provided is to create a second-pass or third-pass listing file of the entire assembly; searching through this far larger document can be tedious as there are usually several references to each undefined symbol. As such, this can easily become a time-consuming process.

The /K option switch is not required if any binary or listing options are specified.

Note: See the description of the /L and /N option switches for the alternate usage of the /K option switch.

Second assembly pass considerations.

A second assembly pass is performed if binary output files are created and/or assembly listing output is produced. During pass two, any additional error messages will be displayed on the system console in a format similar to that of OS/8 PAL8 and TOPS-10 PAL10 [including the nearest symbol plus any numerical offset] to facilitate source code development; additional information may be displayed in certain specific instances.

Programmers may embellish the console output of the assembly process when critical sections are being processed. Several methods are described below:

- a) P?S/8 PAL supports the ERROR directive; this causes a deliberate assembly error. By using the ERROR directive, the programmer can design internal safeguards to prevent logical inconsistencies that might occur during program maintenance. [The design process generally includes the use of conditional assembly to calculate problematic scenarios. As long as certain specific requirements are met, there would be no output of the ERROR directive.]

The output of the ERROR directive is consistent with that of all standard errors during each assembly pass and can include the display of an optional evaluated argument. Automatic binary loading by chaining to P?S/8 BIN will be aborted if the ERROR directive is invoked.

- b) P?S/8 PAL supports the PAUSE directive both as it was originally implemented in PAL III [to ignore the rest of the contents of the latest input file] and also an extension which can embellish console output during the assembly process.

When the PAUSE directive is used with an argument, the argument is evaluated as a 12-bit unsigned value. Output is consistent with the format of standard error message. However, this usage does not report as an error.

The programmer can use the PAUSE directive for a variety of purposes [such as announcing the assembly pass or start of assembly of a critical section]. When a logical inconsistency is detected, the PAUSE directive can be used to output additional information about the error to help maintenance programmers understand the nature of the problem.

Note: The techniques described above and other features unique to P?S/8 PAL can be qualified to prevent use with incompatible assemblers.

The PQS directive is defined only in P?S/8 PAL; this allows the programmer to create conditional assembly statements to restrict usage of these features to P?S/8 PAL and not generate erroneous errors caused by the failure to support the features in other assemblers. An example of this usage follows:

```
IFDEF      PQS  <PQS= 0>  /WILL BE ZERO IF NOT P?S PAL.  
IFNZRO     PQS  <PAUSE .> /DISPLAY CURRENT ORIGIN.
```

Note: The XLIST directive implemented in P?S/8 PAL is consistent with OS/8 PAL8; XLIST statements can be used to hide portions of the assembly as necessary such as assembler-specific statements. Deployed judiciously, all unassembled conditional assembly can be hidden when producing the listing output.

4. Symbol Table Options.

/S The symbol table will be output at the end of the assembly.

If the /N option switch is also set, the symbol table format will be consistent with that of an enhanced listing pass [if enabled].

Note: The exact output may be subject to the number of assembly passes performed; two pass assembly is performed if binary output files are created and/or listing output is produced, or the /K option switch is set.

/A If the /S option switch is set, the entire symbol table will be output at the end of the assembly; this includes all permanent symbols and assembly directives.

Note: If dual-mode [PDP-8 and LINC] assembly is in effect, the symbol table will be set to that which applies at the end of the assembly; certain symbols with dual definitions will display values as used during the final assembly mode.

Certain permanent symbols may have been redefined by the source code; this is especially important in LINC mode assembly where there can be multiple definitions of certain instructions that are model-dependent within the various LINC implementations.

While all default LMODE definitions within P?S/8 PAL are for the PDP-12 LINC variant, source code changes can be made to redefine the affected symbols for compatibility with the LINC-8 or classic LINC as required. [Permanent changes of certain SKP class or other instructions can only be made at the source-code level of P?S/8 PAL.]

5. Listing Options.

/L Assembly listing output will be produced.

Note: Unlike OS/8 PAL8, P?S/8 PAL performs at most two passes; P?S/8 PAL is capable of binary file and listing output simultaneously.

There are obscure quirks of the PAL language when literals are in effect. Novice programmers must avoid certain problematic situations where the definition of certain symbols and values is dependent on the number of passes performed. While this is very bad programming technique, it can be confusing to debug in OS/8 PAL8; PAL8 binary output is created using symbols obtained at the end of pass one possibly updated during pass two. However, PAL8 listing output is created using symbols obtained at the end of pass two and possibly updated during pass three. Unless the programmer is careful to avoid pass-dependent definitions, the binary output may differ from that in the listing output [which may appear correct]. Since P?S/8 PAL always creates all output during pass two, there will not be any pass-dependent differences between binary output and listing output. Programming blunders will be discovered far sooner using P?S/8 PAL [along with additional experience when the improper techniques are realized and then avoided].

/P Enable wide-carriage mode listing output. This option also affects the symbol table output allowing more columns of printed symbols per page.

Note: OS/8 PAL8 only outputs to a meager line width to guarantee compatibility with narrow-carriage devices such as the LP01 despite the fact that many printers support 132 column mode. [Output to the LP01 will likely truncate in many instances of listing output.]

P?S/8 PAL allows far wider source statements than OS/8 PAL8. The listing output of P?S/8 PAL will not be truncated as in the case of PAL8 listing output.

Warning: Users of PAL8 must exercise caution with regard to what are considered "excessively wide" input statements. The number of characters that is considered "excessive" varies with the specific statement type. Depending on the exact circumstances, part of the input statement may be truncated changing the intent of the program. This is not merely a matter of truncating the printed output; rather, part of the source statement may be ignored. Debugging unexpected assembly results may be difficult.

/T Prevent output to the system line printer. All output is directed to the system console terminal. See the description of the /N option switch below.

/N Output "niceties" are applied to the listing output including titles on each printed page. If output is directed to the system console [instead of the system line printer], the output will produce simulated form-feeds by the use of tear-off lines instead of <FF> characters; this will ensure compatibility with system console devices that use roll paper such as the Teletype Model 33.

Note: For systems lacking a line printer that support hard copy console terminals such as the LA36 [which can properly support <FF> characters], the Logical Console Overlay can be configured to redirect line printer output to the same physical device as the as the console output.

The /N option switch enables features that require one extra page of allocated memory. Consult Section 8 of this document for more information regarding memory allocation and related issues.

Title considerations.

When the /L option switch is used in conjunction with the /N option switch, the title field for the entire assembly is taken from the first line of the first input file. Longer assemblies are generally created by concatenating the contents of multiple TFS text files; in most instances, there is no connection between program sections in different files and intended titles on sections of the assembly.

To maintain compatibility with TOPS-10 PAL10 and OS/8 PAL8, P?S/8 PAL supports the change of title field using the first line of every input file. To enable this feature, the /K option switch should be given along with the /L and /N option switches. When each section of a program is contained in an extended-length file [or perhaps a small parameter section in a TFS file], meaningful titles can be provided.

Note: When the /K option switch is used in this context, the alternate usage to force two-pass assembly is superfluous.

As a practical matter, assembly title field changes within an assembly are performed infrequently. For example, the source code of FOCAL, 1969 updates the title field once; as such, two extended-length files [FOCAL and FLOAT] can support the intended title field changes during the assembly.

- /X Enable the creation of cross-reference output by loading the cross-reference module which outputs all symbolic and literal references [if any] as well as the statement number associated with the reference. Cross-reference data is created in statement order during the assembly; at the end of the assembly the data is sorted and output as part of the general listing output.

Enabling cross-reference output causes all listing output to be somewhat wider because all printed lines start with the lowest statement number on the line. [The PAL language allows multiple statements on a line separated by ;.] Statement numbers must be in the range of 1 through 99999.

It is strongly recommended the listing output with statement numbering be targeted for printing on a 132 column printer, especially if the /P option is also enabled.

If the /A option switch is also enabled, all symbolic references will generate cross-reference output; the default is references to user symbols only.

The /X option switch enables features that require two extra pages of allocated memory. Consult Section 8 of this document for more information regarding memory allocation and related issues.

Cross-reference Implementation Restrictions.

Due to changes planned in P?S/8 Version 9A [but not yet implemented], the cross-reference output is currently written to logical unit 7 starting at block 0000; all prior contents of this device will be overwritten. Once the cross-reference output can allocate extended-length files via a user-settable configuration table, this restriction will be lifted; significantly larger programs will be handled than presently on appropriate configurations [such as RK8E/RK05-based systems] due to the availability of multiple extended-length files to store reference data during the assembly.

As of this writing, the processing of the cross-reference output is not fully implemented pending the changes described above with regard to the cross-reference output data; however the statement numbering and creation of the reference data are functional. Logical unit 7 must be available as described above.

Note: OS/8 PAL8 does not support cross-reference output. Instead, OS/8 listing files are rescanned by OS/8 CREF [if enabled] as a post-processing utility. There are several disadvantages to this method of implementation:

- a) The listing output of PAL8 is not a statement-by-statement listing file; statements that were hidden by the use of XLIST and other directives do not appear in the listing. As such, the cross-reference output is potentially incomplete. Statement numbers are inherently inaccurate and are not trustworthy.

By contrast, P?S/8 PAL statement numbering is compatible with TOPS-10 PAL10, which correctly shows statement numbers reflecting hidden statements.

- b) A valid technique for including documentation within PAL source code is to use deliberately failing conditional assembly on a section of commentary as large as an entire page of the listing. Content examples include details regarding program operation and banner pages of large character outlines created from a series of smaller characters; the nature of this form of output is limited only by the programmer's creativity.

Unfortunately, OS/8 CREF has no ability to discern this portion of an OS/8 PAL8 listing file as commentary; as such, the commentary is cross-referenced as if part of assembled code-generating statements. When banner pages are used, there can be numerous junk references to "symbols" that are merely the individual characters used to build up the larger character layouts; as such, the cross-reference output can be cluttered with many erroneous references. Since OS/8 CREF has limitations on symbol size, this can have an insidious effect on the overall usage of OS/8 CREF. See c) and d) below.

- c) OS/8 CREF internally reckons statement numbers as 13-bit integers in the range of 1 through 8192 with an odd quirk. Additional statements wrap back to 4096 instead of 1. As such, there is inherent anomaly in the cross-reference output regarding precise statement numbering of individual references.

By contrast, P?S/8 PAL and TOPS-10 PAL10 allow unique statement numbering in the range of 1 through 99999.

- d) OS/8 CREF has limited overall capability. If the entire cross-reference cannot be accomplished in one pass the program aborts unless the /M option switch is used. If the /M option switch is enabled, CREF operates as a strictly-defined two-pass operation which may [eventually] allow the program to complete. However, each "half" of the overall operation must succeed to obtain a successful overall result.

Note: A quirk of the two-pass method is the insertion of an extraneous <FF> in the output stream when the second pass starts. [This is documented but not explained.]

The split-point for this so-called "mammoth" mode is set to the theoretical symbol LGNNNN. References to all symbols that sort lower than this value are scanned in the first pass while ignoring references to symbols that sort higher. The second pass processes all symbols that sort higher than LGNNNN including all references to current-page and page zero literals; the references processed in the first pass are ignored. Each pass must scan the entire listing file.

Because of the arbitrary nature of the split-point, it is possible for the overall cross-reference to fail because the program symbols are not "balanced" sufficiently to conform to this arbitrary scheme [for which there are no options].

- e) Because of the inherently redundant nature of rescanning text files attempting to obtain information about the assembly without having access to the original symbol table and parsing techniques, OS/8 CREF operates very slowly.

By contrast, assembler-based cross-reference implementation has the advantage of only processing legitimate references to symbols and literals; commentary is ignored. Instead of rescanning entire listing files, the essential cross-reference data is scanned as many times as necessary to process all references; there is no arbitrary static split-point nor requirement of at most two passes on the far smaller set of data.

Note: Many large PAL programs cannot be completely processed by OS/8 CREF. This includes the source code of P?S/8 PAL.

6. Literal and Link options.

- /Q Enable generation of current page and page zero literals. Link generation is disabled; attempts at off-page references will be treated as errors.
- /O Enable generation of current page and page zero literals. Link generation is enabled; however, links will be flagged as errors.

Enabling literal generation and/or link generation requires three extra pages of allocated memory. Consult Section 8 of this document for more information regarding memory allocation and related issues.

Using the /Q option switch in conjunction with the /O option switch will suppress error messages on generated links. However, the count of links generated is always included in the statistics page produced at the end of the assembly.

Assembly listing lines containing generated links will be flagged with 'L'; this feature cannot be disabled.

Note: OS/8 PAL8 is compatible with PDS/8 PAL with regard to all aspects of the disposition of generated link statements. However, all features regarding literals in PAL8 cannot be disabled. See below for detailed warnings regarding generated links that apply to both assemblers.

- /E Reset the current page literal extent when leaving the latest page. [Use of the /E option switch requires some combination of the /Q and /O option switches to be meaningful.]

Note: See below for more information on literal extent issues and options.

Limitations of Generated Links.

The usage of generated links should be totally avoided; in general, links do not work!

While this caveat ought to be sufficient, a study of the PDP-8 architecture reveals the details of the problem:

Links are an attempt to pervert the PDP-8 architecture into something it isn't, yet have sloppily written programs work while violating the hard-wired rules of the computer's instruction set.

A link is an indirect reference fixup where the program specifies a direct reference to memory and instead an indirect reference is manufactured by the assembler involving a literal address pointer not specified in the source program.

The following is an attempt to violate the rules of the PDP-8 instruction set flagged as an error [because link generation is not enabled]:

*200

/ON PAGE 1

TAD 577

/ATTEMPT TO ACCESS PART OF PAGE 2

This usage is impossible because unless the reference is to a page zero location, the only allowed value would be in the current page range of 0200-0377 in this instance; 0577 is part of page 2. Locations on any page from 2 [0400-0577] through 37 [7600-7777] are equally inaccessible. The instruction is correctly marked with the **IR** error message [Illegal Reference].

If link generation is enabled, the same source code instead produced flagged output. The instruction at 0200 is changed to TAD I 377; additionally, location 0377 is generated containing 0577. Ignoring for the moment the instruction is now an indirect reference to 0577 using the pointer address 0377 as an intermediary, to the naive programmer, the intended logic is satisfied: the contents of location 0577 are added to the accumulator.

Note: In all of these examples, symbols are not present to cloud the issue. Actual attempts at link generation are likely completely symbolic; had the code been located in an accessible location, there would be no links required; links often come about when code overflows the intended page into the next page. By using purely numerical references, we are forcing the results to be clear.

To the naive programmer, the program "works" per se. Unfortunately, in real programming, considerations outside of this example tend to crop up; programs that had "worked" suddenly don't work at all and get unavoidable assembly errors. [The kind of error enabling link generation cannot remedy.]

The naive programmer might be tempted to use a location already inaccessible [due to being on another page from the instruction] as a pointer. The assembler will properly mark this with the error message **II** [Illegal Indirect]. Since it takes a pointer to access indirectly the address of the pointer, it is not possible to use it indirectly. Had the programmer obeyed the rules of the instruction set, this would come as no surprise.

Before investigating the insidious nature of link generation failing to work when no error messages occur [just generated link flagging], it will be shown that *certain* forms of generated links actually work, as long as the mechanism is not "stretched" to attempt to perform the impossible.

In all cases, links involving *direct* JMP or JMS instructions do "work" in the sense described above. [Statements of the form JMS I or JMP I cannot function as explained above.] If this were the only issue, lazy programmers would have a valid excuse to desire link generation.

Unfortunately, all of the other forms of link generation will fail depending on the exact nature of the running program; the assembler cannot have knowledge of the programmer's intentions in the dynamic sense.

The following is an example of a generated link that will be flagged as a link [and will not generate an assembly error if error messages on links are suppressed], yet the program logic will fail:

[This is the same program as above; the program is started from the front panel with the instruction field [IF] set to 0 and the data field [DF] set to any value from 1 through 7]

*200

/ON PAGE 1

TAD 577

/ATTEMPT TO ACCESS PART OF PAGE 2

The program does not accomplish the intended function. The contents of location 0577 in the data field are added to the accumulator.

In general, programs are written for the 32K PDP-8 address space. The data field is often part of some internal convention of the program logic. When the program uses direct references as the nominal statements do in the example, the data field plays no role in the logic of the program. Programmers who never enable link generation are certain their programs are designed as expected as long as no statements are marked with errors during the assembly. [Note: This is not to say there are no logic errors in the program. But at least the type of error illustrated here won't be present.]

Thus, as soon as extended memory factors creep into the logic of a program dependent on link generation, insidious failures start to occur. A naive programmer who never learned the necessary discipline required to avoid this programming pitfall is now faced with an unsolvable dilemma. The only possible repair is to "bite the bullet" and eliminate all generated links as a first step towards debugging the program; eliminating the attendant error messages solves at least 99% of the problem. [Note: Cleaning up after link generation might involve a lot of work all at once; had links never been used, the work to eliminate assembly errors would have been accomplished in small increments.]

In short, link generation "works" only if direct JMP and JMS instructions are involved. Link generation fails to "work" if AND, TAD, ISZ or DCA instructions are involved. All forms of indirect usage attempts always fail.

Positing a stubborn and lazy PDP-8 programmer [the author has met people who fall into this category; their careers as PDP-8 programmers who use generated links are always short], one of those who insist on using link generation despite all the caveats. The excuses are invariably of the form:

- a) Only uses JMP and JMS links. [Don't believe this for a minute.]
- b) Never uses the cases that cannot work and counts how many links are generated in the assembler statistics page every time to make sure nothing has changed.

There are several logical fallacies in a) and b) above:

- c) One can only assume a) is true; it is necessary to check the entire program to ensure this is the case. Confirming this is far more work than just avoiding generated links.
- d) The count of links generated may be the same as past usage. However, there is still no guarantee all of the generated links avoid the impossible cases; if the code is moved around sufficiently, a JMP or JMS link could drop out and one of the impossible cases might replace it in the assembler link count statistics. It is still necessary to check the entire program as in c) above; the description of the extra work involed still holds.
- e) In reality, people this stubborn are hypocritical. They also tend to use the potentially impossible form of generated links because they "know" the data field will "never" be wrong. This puts them squarely into the insidious problem position described earlier.
- f) When cross-field calling conventions are established in meaningful programs, it is belatedly understood just how problematic the artifice of link generation really is. The work involved in cleaning up the program can be profound.
- g) Other architectures may support something that can be likened to "training wheels". Generated links already take programmers "off the rails".
- h) In the case of P?S/8 PAL, TOPS-10 PAL10 and OS/8 PAL8, additional option switches have to be enabled to suppress error messages associated with link generation. This is by design, as the developers of real operating systems never use generated links and are trying to guide programmers to success in PDP-8 programming. Hopefully all stubborn and lazy programmers eventually get past this illogical stage.

Examples of Literal Statements.

P?S/8 PAL supports the two general forms of literal generation along with a unique third form that can shorten program development time.

- a) Current page literals - The operand address is located on the current page of the program; the address is assigned by the assembler starting at relative location 177. Additional literals are created in reverse order; a second current page literal is placed at relative location 176.

The maximum number of current page literals allowed is 63 [decimal] in the relative range of 101-177. When a statement contains a current page literal, the current page literal table is searched for a matching value. If successful, the existing literal address is used. New values create additional current page literals. When either too many current page literals are attempted or statements on the current page overflow into the literal table, a **PE** [Page Exceeded] error message is displayed.

A typical current page literal usage follows:

```
TAD (3) /ADD 0003 TO THE ACCUMULATOR
```

- b) Page zero literals - The operand address is located on page zero; the address is assigned by the assembler starting at location 0177. Additional literals are created in reverse order; a second page zero literal is placed at location 0176.

The maximum number of page zero literals is 112 [decimal] in the range of 0020-0177. [Locations lower than 0020 are disallowed due to the special characteristics of these locations; locations 0010-0017 auto-increment when referenced indirectly.

When a statement contains a page zero literal, the page zero literal table is searched for a matching value. If successful, the existing literal address is used. New values create additional page zero literals. When either too many page zero literals are attempted or statements on page zero overflow into the literal table, a **ZE** [page Zero Exceeded] error message is displayed.

A typical page zero literal usage follows:

```
TAD [3] /ADD 0003 TO THE ACCUMULATOR
```

- c) Conditional literals - The operand address is either on the current page or on page zero depending upon specific circumstances. If a matching page zero literal value exists, it will be used. Failing a page zero match, If a matching current page literal exists, it will be used. Failing both a page zero match and a current page match, a new current page literal is created [subject to the possibility of a **PE** error if the current page overflows].

A typical conditional literal usage follows:

```
TAD  \3/                /ADD 0003 TO THE ACCUMULATOR
```

The actual code generated is dependent on the literal pools in effect at the point in the assembly the usage is encountered.

A common technique employed in well-developed programs is the use of source code libraries of commonly used subroutines. It is desirable to use page zero literals where appropriate; however, page zero literals should not be used if there are only singular references. By using conditional literals, more effective management of memory is achieved without the need to hand-optimize the library source code statements containing literals.

Note: This feature is not supported by OS/8 PAL8.

Literal Extent Issues.

In general, all literals associated with any current page should be defined in proximate order to the code statements referencing them. This facilitates locating any potential matches to avoid new literals [if possible].

Leaving the present page, the existing literals are dumped to finish the intent and requirements of all source code statements defined so far.

If that page is ever revisited, it is possible to define additional literals on the page because the literal extent formerly in effect was saved and will be restored. However, since the actual literals have been dumped, all new literals will only be able to search for a match within the newly-created literal pool.

Note: This feature, while enabled by default, is hardly ever used. It is supported primarily to maintain compatibility with the literal generation features of TOPS-10 PAL10 and OS/8 PAL8.

If the /E option switch is used, the literal extent when leaving a current page is totally reset; no history of previous literals on any current page is retained. [Note: The use of the /E option switch does not impact page zero literals, which are completely reset upon either end of assembly or a FIELD directive. In some documents about PDP-8 assemblers, this is referred to as "forgetting" current page literals.]

Resetting literal extents is useful when defining overlays. The overlay code must be created in the context of another section of the assembly which presumably will be replaced in memory by the overlay code; however, the overlay code must maintain its own current page literals as required.

while there are many techniques associated with the deployment of overlays, the /E option switch is always required to allow overlays to maintain their own current page literal pools.

Note: Future implementations of P?S/8 PAL [or a new assembler for the P?S/8 SHELL based on P?S/8 PAL tentatively to be called XPAL] will likely invert the default condition regarding literal extent retention due to disuse of the current default; as a practical matter, sections of program code that revisit formerly used pages are always part of an overlay structure. There are no justifiable reasons to deliberately separate small program sections on the same code page.

7. Dual Assembly Mode Options.

- /8 Enable LINC mode assembly; the default operating mode is PDP-8 mode [PMODE]. While not strictly necessary, it is good practice when enabling dual assembly to state the desired opening mode; this is equivalent to using the PMODE directive. The current location counter is set to the usual PDP-8 value of 0200.
- /9 Enable LINC mode assembly; the default operating mode is LINC mode [LMODE]. While not strictly necessary, it is good practice when enabling dual assembly mode to state the desired opening mode; this is equivalent to using the LMODE directive. The current location counter is set to the usual LINC value of 4020.

Enabling the dual mode assembly module requires the allocation of three additional pages of memory. Consult Section 8 of this document for more information regarding memory allocation and related issues.

Dual Mode Assembly Tips.

While it is good practice to start source code with the appropriate mode directive, it is possible [using conditional assembly techniques] to determine the current assembly mode. References to the current definition of the HLT instruction [which will be 7402 in PMODE and 0000 in LMODE] can determine whether LMODE or PMODE is in effect. Critical programs can abort the assembly if it is determined the assembly process is not in keeping with source code requirements [which could include failure to enable LINC mode assembly].

P?S/8 PAL is the only known assembler that fully supports LINC mode assembly, literals, large assemblies and the full complement of PDP-8 assembler features. The LINC mode support is generally compatible with the LAP6-DIAL/DIAL-MS assembler; however, all versions of DIAL can only run on the PDP-12; additionally, the DIAL assembler does not support literals. P?S/8 PAL does not require LINC hardware to assemble LINC mode programs.

While the use of literals generally applies to PDP-8 code, certain features can be adapted to LINC mode programming with restrictions. Advanced users can contrive viable cases modestly worth pursuing.

In general, LINC programming does not require literals due to significantly different addressing modes. Immediate mode addressing replaces literals because the immediate mode operand cannot be used in other instructions. Immediate mode specifying the address of common variables can be likened to literals; if the addresses can be contrived to conform to PDP-8 page addressing, actual PDP-8 literals can be used obtaining the intended results. [Note: Any form of arithmetic used to calculate addresses in LINC mode is performed using one's complement arithmetic.]

All LINC instructions on the PDP-12 and the LINC are implemented in hardware. On the LINC-8, certain instructions are trapped and emulated using a support program known as PROGOFOP [PROGRAM OF OPERATION]. PROGOFOP is required for 100% emulation support; however, short sections of LINC programming can be initiated by PDP-8 programming with interrupt-handling optional. [Note: The PDP-12 also supports an alternate form of trap simulator to allow programs written for the classic LINC and/or LINC-8 to be run as intended, perhaps with alternate interpretation of the LINCtape class instructions. For more information, consult DEC documentation of the PDP-12 trap simulator.]

LINCtape Information.

P?S/8 and OS/8 run on 128 words/block LINCtapes; DIAL uses extended-length 256 words/block LINCtapes. Utilities to access 256 words/block LINCtapes require custom programming for each machine. To transfer assembled programs to the classic LINC requires a custom image utility as conventions for LINC operating systems vary. LINC systems cannot access PDP-8-oriented LINCtapes; the LINC-8 can handle all formats because the underlying hardware is actually a PDP-8 peripheral. The PDP-12 includes extended operations to work with all common formats.

8. Memory Management Options and Issues.

/C Enable internal reconfiguration of assembler resources; using the /C option may make certain assembly configurations on smaller machines viable at the expense of performance. See below for additional configuration details.

P?S/8 PAL is designed to run on 4K or larger machines; additional memory, if available, is used to allow a larger symbol table which is generally the limiting factor with regard to overall program size that can be successfully assembled.

When P?S/8 PAL is first loaded, the passed option switches are checked to determine the running configuration size with regard to modular assembler components once the assembly process begins. Deselected modular sections are removed; retained modular options are then relocated as required to maximize the space available for symbol table and input/output buffers.

Depending on the options selected, it is possible the symbol table load could overflow the assembler capabilities; the configuration selected cannot run given the normal allocation of input/output buffers.

To aid in achieving a viable configuration, the number of input/output buffers can be reduced; the space is then made available for additional symbols. The assembly process will take longer due to the smaller buffers [but will generally be successful unless the source code has many symbols].

To gauge the magnitude of the problem on 4K systems, the following is a list of the extra memory allocation requirements [over the minimum needed for an assembly without any modular sections retained]:

- a) /N-related "niceties" options [fully formatted headers on listing particulars] requires one additional page.
- b) Literal and/or link generation requires three additional pages.
- c) Enabling dual mode [LINC] assembly requires three additional pages.
- d) Enabling cross-referenced listing output requires two additional pages.

Each page retained for symbol table usage adds the ability to support 32 additional symbols; an assembly with no modular options retained can support 288 additional symbols.

If the /C option switch is set, the input and output buffers are reduced to a minimum complement causing the assembly process to take considerably more time; however, sufficient memory space should be available to support any combination of the optional modular program sections as described above.

P?S/8 PAL is designed to dynamically take advantage of additional memory if available. On all systems with 8K, at least another 3K of memory is available for the symbol table; as such, all modular components can be retained without restriction.

Note: Certain P?S/8 configurations reserve the highest quarter memory field available for the system device handler extension. For all systems that do not require the handler extension, the full additional 4K of memory is available to P?S/8 PAL [and other programs].

When 7K [or more] memory is available, symbols are not stored in field 0. The /C option switch can be used to sacrifice performance while allowing additional symbols residing in field 0.

The maximum number of symbols allowed is 4095 including all permanent symbols and directives. If 16K of memory is available for symbols [20K total memory or more], there is no point in enabling the /C option other than to perform timing tests to measure the resulting performance degradation; the exact configuration with regard to the modular options becomes moot for the larger configurations.

Note: OS/8 PAL8 requires 24K or more to support the maximum symbol table size of 4095 symbols due to certain design limitations of OS/8.

P?S/8 PAL Assembler Performance.

P?S/8 PAL performance was once compared to OS/8 PAL8 using a deliberately biased configuration and a stopwatch. While P?S/8 PAL took twice as long to assemble a large file chosen at random from various OS/8 source programs under development at the time, the P?S/8 configuration was based on TC08/TU56 DECTape while the OS/8 configuration was based on the RK8E/RK05 removable cartridge disk [which is inherently a higher performance system]. Given the overhead of the underlying system devices, it is estimated that P?S/8 PAL can assemble large programs as much as four times as fast as OS/8 PAL8 on identical hardware configurations.

Note: The performance of the P?S/8 BIN *slurp* loader was compared to OS/8 ABSLDR on the same configuration. The DECTape-based P?S/8 *slurp* loader successfully loaded binary files twice as fast as OS/8 ABSLDR on the RK8E/RK05.

9. Additional Binary Chaining Options.

Various options of the P?S/8 BIN utility apply during a successful chain operation. Assuming no assembly errors, chaining to P?S/8 BIN is generally used for the purpose of loading the binary output files created during the assembly.

Note: See the description of the /W option switch below for the alternate purpose of chaining to P?S/8 for the purpose of punching binary paper-tapes.

As explained above, chaining to P?S/8 BIN creates a list of binary input files derived from the list of output files created during the assembly; this list may be the direct result of explicitly passed output files or as a result of some particular combined usage of the /B, /D and /U option switches. As necessary, the list may have been truncated if excess output files were specified.

The following additional options of P?S/8 BIN apply if set in the original P?S/8 PAL command line:

- /G Chain to P?S/8 BIN for the purpose of either loading the binary files created during the assembly or punching binary paper-tapes of the binary files [if the /W option switch is also set]. The chain is inhibited if there are assembly errors.
- /V Enable the virtual binary file loader instead of the hardware-specific binary file loader [if relevant].

Where possible, P?S/8 BIN loads binary input files using an implementation of the *slurp* loader as originally defined by Richard Lary for the R-L Monitor System. The P?S/8 implementation supports loading anywhere within the 32K PDP-8 memory space [restrictions apply to system reserved memory areas].

Specifics of the P?S/8 system configuration will indicate whether the *slurp* loader is available [as only certain hardware configurations are capable of supporting the *slurp* loader].

On systems that do not include a hardware-specific *slurp* loader, a virtual binary file loader is used that requires the presence of the system device handler. Other than certain performance issues and minor tradeoffs, the two loaders perform the same overall function.

If the /V option switch is set, the virtual binary file loader is used instead of the hardware-specific *slurp* binary file loader. This is a user preference option assuming the hardware-specific *slurp* loader is available; it may be preferable to force the virtual loader under certain circumstances.

- /I If the *slurp* loader is in effect, the system device handler is reloaded after all binary files are loaded. As a restriction, it is not possible to pass the maximum input file count of 17 input files; the maximum input file count is reduced to 16 [which is generally of no consequence to most users].

Note: If the virtual loader is in effect, the /I option switch is ignored; the restriction on maximum binary input files does not apply. If the *slurp* loader is in effect and the /I option switch is not set, the system device handler will not be available; however, a hardware-specific bootstrap will be placed into locations 07600-07642 to allow program exit to 07600 in the identical manner as if the system device handler were present.

- /H All user memory is preloaded with 7402 [the PDP-8 HLT instruction] before binary file loading. This is strongly recommended when debugging programs undergoing development to prevent unexpected loss of control.

- /Z All user memory is preloaded with 0000 [the LINC HLT instruction] before binary file loading. This is strongly recommended when the program under development contains numerous LINC instructions or it is expected the binary output will be saved in core-image format; unloaded locations will show 0000 in any form of memory or storage block dump printout.

P?S/8 can create a core-image of a program under development using the GET /Z command [or as a result of a chaining operation as described above]. The resultant core-image blocks can be transferred to pairs of TFS files which can be restored to the % and \$ file for execution.

Future releases of P?S/8 will include the SHELL overlay [currently under development] which will support versions of the ENCODE and DECODE programs which are currently available as part of the Kermit-12 program collection. Core images created in the P?S/8 SHELL environment [through equivalent means to the GET command] passed through the ENCODE program will produce smaller output files due to the elimination of uninitialized storage locations within the core-image file.

If the =xxxx passed numerical option is set, the value xxxx is used as the starting address for the program; the default starting field is 0. If the program will be started after loading, the following additional option switches apply:

- /1 The program will start in field 1 at location xxxx as specified in the =xxxx passed numerical value.
- /2 The program will start in field 2 at location xxxx as specified in the =xxxx passed numerical value.
- /3 The program will start in field 3 at location xxxx as specified in the =xxxx passed numerical value.
- /4 The program will start in field 4 at location xxxx as specified in the =xxxx passed numerical value.
- /5 The program will start in field 5 at location xxxx as specified in the =xxxx passed numerical value.
- /6 The program will start in field 6 at location xxxx as specified in the =xxxx passed numerical value.
- /7 The program will start in field 7 at location xxxx as specified in the =xxxx passed numerical value.

Note: All of the options stated above assume the /W option switch is not set to allow binary file loading.

If the chain to P?S/8 BIN is for the purpose of punching binary paper-tapes, the following option switches apply:

- /W The /G option switch is set; the chain to P?S/8 BIN will be used to punch binary paper-tapes if there are no assembly errors [else the chain to BIN will be prevented].
- /F The default device for the binary paper-tape output is the low-speed console device paper-tape punch found on various console terminals such as the Teletype Model 33 ASR and Model 35 ASR [device 04]. If the /F option switch is set, the binary output is directed to the high-speed punch [device 02].
- /R By default, the paper-tape output is punched in BIN format. If the /R option switch is set, the output format used is enhanced RIM format [which includes a checksum so the binary paper-tape can also be read by the BIN loader].

Note: the /0 option switch is not passed to P?S/8 BIN as it has alternate meaning within P?S/8 PAL. See Section 11 for more details.

10. Text Management Options and Issues.

- /J Prevent generation of a 12-bit word of 0000 after text strings with an even count of characters as used with the TEXT and SIXBIT directives.

Text Directive Issues.

The PAL language defines the TEXT directive which is used with a string argument. The string consists of an indefinite number of printable characters preceded by a delimiter character which cannot be included in the generated text; the string ends with a second usage of the delimiter character.

Note: P?S/8 PAL does not require the trailing delimiter character unless there are multiple statements on the same line.

Early implementations of the TEXT directive restricted the delimiter character to ", ' and in some cases @ [which is not supported by P?S/8 PAL and other assemblers]. The use of " or ' as text delimiters is recommended for maximum compatibility with other PAL assemblers; P?S/8 PAL and OS/8 PAL8 support an extension allowing any printing character not contained within the text string to be used as the delimiter character.

All usage of the TEXT directive creates strings of six-bit ASCII text characters packed two per 12-bit word. If the count of characters in the string is an odd number, the last 12-bit word has 00 in the low-order six bits which can act as a string delimiter.

Since the most likely usage of the TEXT directive is to generate strings used with printing subroutines, the default action for string arguments with even character counts is to output an additional 12-bit word of 0000 to ensure the presence of a string delimiter.

TEXT directive usage is not limited to string creation as described above; some alternate schemes use internal printing characters as delimiters or other control purposes. Within the P?S/8 keyboard monitor coding, explicit string length arguments are used in some instances; in other cases the string length is an implicit constant value. The 00 case plays no role in any of these cases; creation of an extra 12-bit word of 0000 is undesirable.

Note: Early PDP-8 programs such as FOCAL, 1969 were developed on systems incapable of modifying the output of the TEXT directive; as such, the source code includes statements to backup the assembler current location counter to allow the desired contents which differs from the automatically generated 0000 value. Ironically, the source-level workaround is incompatible with the /J option switch which would eliminate the problem.

As documented above, setting the /J option switch prevents the creation of the extra 12-bit word of 0000 on even count text strings. It is strongly recommended the disposition of the /J option switch be documented at the beginning of all source code; in many cases, this may be the only departure from assembler default settings and could easily be overlooked.

Other Text-related Issues.

P?S/8 PAL supports the SIXBIT directive which takes the same general form as the TEXT directive. The only difference is the internal six-bit code used is not the PDP-8 standard six-bit subset of the ASCII character set. Instead, the alternate definition of six-bit text as used on DEC 18-bit and 36-bit computers applies.

The basic difference between the two character sets is the highest of the six bits used is inverted. Input processing of seven-bit ASCII characters to six-bit code is easier using the PDP-8 standard while output processing of six-bit code to seven-bit ASCII characters is easier using the alternative standard. The PDP-8 standard was chosen largely because there are often multiple input routines while output is generally performed by a single output routine.

Note: OS/8 PAL8 does not support the SIXBIT directive.

The DEVICE and FILENA{ME} directives are supported by P?S/8 PAL and OS/8 PAL8. These are special-purpose text directives to create a four-character and eight-character text string respectively. Early OS/8 programs required the use of the /J option switch [or its equivalent in OS/8 PAL8 which is the /F option switch]; this allows creation of the intended string arguments while preventing compatibility with the usual string printing applications, which require even-length string termination. With the availability of the DEVICE and FILENA{ME} directives, the normal flexibility of text string creation is restored.

Note: P?S/8 supports the FILENA{ME} directive for compatibility purposes only. Future P?S/8 systems will support the P?S/8 SHELL with an extended assembler based on P?S/8 PAL to be known as XPAL. XPAL will support an option to extend the FILENA{ME} directive to support SHELL directory file names.

Certain user-modified assemblers exist that support the TEXTZ directive which forces the extra 12-bit word of 0000 even if the /J option switch [or equivalent] is in effect. Future versions of P?S/8 PAL may support the TEXTZ directive and the SIXBTZ directive [using the alternate six-bit text code].

Note: As of PAL8 Version B0 [the last version released by DEC], the TEXTZ directive is not supported.

Most PAL assemblers [including P?S/8 PAL and OS/8 PAL8] support the `"` operator in expressions. The next character [which cannot be `<HT>`] is used to form the seven-bit ASCII value of the stated character; the mark parity bit is set.

A proposed ASCII directive would take a string argument much like the TEXT directive to create a string of seven-bit ASCII characters. The /J option switch would prevent a trailing 12-bit word of 0000.

An additional ASCIIZ directive could be implemented to force the creation of the trailing 12-bit word of 0000 regardless of the setting of the /J option switch.

Note: An additional issue is whether to have an option to prevent the mark parity bit within the text string as arithmetic operations are not possible on individual string characters.

Even without any form of ASCII directive, it is currently possible to create seven-bit ASCII text strings with any of the discussed variations using the `"` operator. When used alone, each character in the string will have the mark parity set; string delimiter characters can be added if desired, such as ending the string with a 12-bit word of 0000 or making the last character of the string negative [in the 12-bit sense]. By adding an additional element in each expression, it is possible to strip off the mark parity bit:

```
HLOWRLD,"H&177;"E&177;"L&177;"L&177;"O&177;" &177 /HELLO WORLD MESSAGE
"W&177;"O&177;"R&177;"L&177;"D&177!4000 /NEGATIVE ENDS TEXT.
```

An unnamed additional text directive has been proposed to create packed seven-bit ASCII text strings following the "three for two" packing convention as used in OS/8 text files [and the P?S/8 SHELL]. Because of alignment restrictions, additional null eight-bit bytes will be required, regardless of the issue regarding the mark parity bit. If the string is meant to be an image of an actual file section, there must also be a mechanism to append a trailing `Control-Z` character into the string.

Note: The issue of a "three for two" directive will likely be taken up in the XPAL assembler which will support upper-case/lower-case input files. A PARITY directive with appropriate argument support can be implemented to handle all mark parity bit [and perhaps `Control-Z` character] issues raised above.

11. Additional Option Switches and Issues.

/O Automatically generate *200 after the use of the FIELD directive.

Note: Most programs have data or explicit origins after FIELD settings. By using an explicit origin setting, a known serious bug of one of the released DEC BIN loaders [known as the "self-starting" binary loader] can be avoided.

The problem with this version of the loader [which was never fixed] is that when a FIELD setting is encountered, it is applied prematurely; the last data word from the previous address [presumably intended for use with the former FIELD setting] is erroneously loaded into the new field. By ensuring the last data frame is an origin setting, the problem is generally avoided.

As a practical matter, most well-designed programs use specific origins after FIELD settings for appropriate purposes; as such, this option is generally irrelevant.

/Y The ! operator is changed from inclusive OR [.IOR] to shift left six bits.

without the /Y option switch set, 22!33 generates 0033; with the /Y option switch set, 22!33 generates 2233.

The ! operator [with the /Y option switch set] can be used to create short six-bit text expressions:

A= "A&77

B= "B&77

A!B

Each word created using the ! operator in this manner produces the same result as a 12-bit component [two characters] of a text string.

Note: Certain programs require the default ! when embellishing combined OPR or IOT instructions:

CLA!CLL!CML!RTL /SET THE ACCUMULATOR TO 0002

DTSF!DTRB /SKIP ON DECTAPE DONE OR ERROR FLAG
 /SET; READ THE ERROR REGISTER

Programmer preference will determine whether to use the /Y option switch. As in the case of the /J option switch, if the /Y option switch should be set, this should be documented in the beginning of the source code to ensure proper assembly.

12. P?S/8 PAL Directives and Issues.

P?S/8 PAL is fully compatible with all standard directives as implemented in OS/8 PAL8. Certain additional directives are supported compatible with other assemblers such as the DIAL-MS assembler or TOPS-10 PAL10. Additional directives are unique to P?S/8 PAL.

A complete list of P?S/8 PAL directives is provided in Appendix B.

In one particular case, the DTORG directive, the OS/8 implementation is deliberately not supported in P?S/8 PAL. A proposed solution to the underlying problem is presented below which will allow the DTORG directive to be useful to P?S/8 system generation in future releases of P?S/8 PAL.

OS/8 PAL8 Implementation of DTORG.

As implemented in OS/8 PAL8, the DTORG directive is provided for compatibility with TOPS-10 PAL10. The DTORG directive is meant to create 128 words/block numbered block references within the binary output to allow a special-purpose loader [only used by the Typeset-8 group] to write Typeset-8 system blocks on DECTape [and perhaps other devices].

The general function is to specify a numbered storage block on a TC01/TC08/TU55/TU56 DECTape followed by the [up to] 128 words to be written to the specified block.

The origin in memory is set prior to the DTORG directive; however, it is generally ignored because no form of standard loader [such as the BIN paper-tape binary loader] is supported. The origin is used to define the upper limit in memory of the [up to] 128 data words that follow; once an entire PDP-8 page has been defined [placing the current location counter on the next PDP-8 page], the intended block contents are finalized.

Typeset-8 programs create a series of these small sections; each is loaded by a special-purpose Typeset-8 generation utility which writes the data to the specified DECTape [or other device] block. However, there are serious issues:

- a) Each section written to a targeted DECTape block is taken from a lengthy punched binary paper-tape created at an OS/8 development site; the tape invariably was the latest release of a major section of the Typeset operating system [or one of its utilities]. Typeset-8 machines generally did not have high-speed readers. In-house Typeset-8 computers invariably used Teletype Model 35 ASR terminals to read in the binary data.

- b) There is no form of checksum protection associated with the written data. Since the binary paper-tapes were generally quite long, this is patently foolhardy.

By the nature of paper-tape, it is expected that reading in long binary paper-tapes will occasionally result in read errors. This is why the BIN loader format includes checksum frames.

The RIM loader, which lacks a checksum mechanism, is only used to load short binary programs such as the BIN loader, which in turn can load in significant binary program data with the confidence provided by passing an overall checksum test.

- c) Despite being created by PAL8 in OS/8 standard binary format [as a .BN file], the binary output violates the OS/8 binary file standard; the output of a program that uses the DTORG directive cannot be loaded successfully by OS/8 ABSLDR. Attempts to read in the punched binary paper-tape will always end in a checksum error, even if an error-free read was performed. This is because the implementation of the DTORG directive usurps the underlying BIN format that OS/8 converts into three-for-two OS/8 records [384 frames per logical record].

As such, it is impossible to improve the performance of the Typeset-8 special-purpose loader; reading in the entire binary paper-tape will not yield valid checksum information that could be used to reject attempts to read the lengthy tapes that introduce additional flaws.

Proposed Remedies to the DTORG Problem.

Apparently due to internal DEC politics, no attempt was made to remedy this problem by educating the Typeset-8 programmers regarding their naive design; various well-known mechanisms can easily solve the problem.

- a) Instead of implementing the DTORG directive as a violation of the internal binary format of BIN utilities and OS/8, the binary data can be patterned after a variety of utility formats such as certain details of P?S/8 non-system device handlers and OS/8 device handlers [as required by OS/8 BUILD].

An arbitrary series of origin settings can be used to indicate the intention of DTORG directive statements:

```
DTORG 1234 /DESIGNATE BLOCK 1234 FOR OUTPUT.
```

The DTORG usage could create the equivalent of the following binary output:

```
PREV= . /NEED THE PREVIOUS ORIGIN SETTING.  
*7777;*1234;*PREV /OUTPUT FLAG ORIGIN FOLLOWED BY  
/THE DTORG BLOCK NUMBER FOLLOWED  
/BY THE PREVIOUS ORIGIN.
```

A special-purpose loader is still required to create Typset-8 system blocks in the proposed format; however, the binary data created is compatible with all loading mechanisms; a lengthy binary paper-tape punched out and then read on a Teletype Model 35 ASR read station could be known to have passed the checksum test. Afterwards, all designated blocks can be written out at once. [Note: The Typeset-8 system runs on 4K machines. The binary paper tapes used would tend to be somewhat longer than the binary paper-tape for FOCAL, 1969 as a representative example.]

- b) The example remedy represents the most expedient method to solve the DTORG problem. More elaborate schemes could include additional origin settings, perhaps following the last two data-based origins as shown in the example. An overall mini-checksum for the entire string of origin settings could be created if desired.

Regardless of implementation details, the entire problem of supporting incompatible binary files would be eliminated. All binary files can be used with all available binary format utilities. After punching the lengthy paper-tape, it could be read back in using the high-speed paper-tape reader. The resulting file would not only have to pass the standard checksum test, but additionally could be directly compared to the original binary output file to completely confirm the validity of the paper-tape before bring the tape to the Typeset-8 site for the usual purpose of generationg Typeset-8 system components.

- c) without having to support incompatible binary format, the need to store paper-tape binary frames becomes superfluous. OS/8 could have easily been developed using a more efficient internal file format similar to P?S/8.

It is well known that DEC managers perverted certain aspects of the demo project that was eventually to become OS/8. Their demonstrable ignorance of PDP-8 operating systems has left its mark on various forms of internal poor design aspects of this system.

It is surmised that Richard Lary intended to use *slurp* binary format for OS/8 [having just recently demonstrated the R-L Monitor to the same dim bulbs who rejected it], but the edict to use images of binary paper-tape frames clearly was created in part due to demands of the Typeset-8 group.

Note: TOPS-10 PAL10 also implements the creation of incompatible binary paper-tapes; the DTORG support is identical to OS/8 PAL8.

- d) OS/8 PIP could easily be modified to convert compatible binary files with DTORG usage within [as proposed above] into Typeset-8 format binary paper-tapes [assuming no cooperation from the Typeset-8 group]. Thus, the problems associated with incompatible binary format would only affect the users of Typeset-8, and not the entire PDP-8 community at large.

while not likely to occur, the binary format of OS/8 utilities could be updated to *slurp* binary format which would improve the performance of OS/8 for all users.

- e) while the DTORG directive is not particularly useful for OS/8 purposes [just about every aspect of the present DTORG implementation is responsible for some negative aspect of OS/8 performance] since it addresses 128 words/block storage [while OS/8 addresses 256 word logical records], it can be useful for future implementation of P?S/8 system generation utilities that follow the overall flow of the Typeset-8 system generation method [but none if the flawed details].

Future releases of P?S/8 PAL [and the SHELL assembler XPAL] will develop a more useful implementation of the DTORG directive; for the present, P?S/8 PAL merely implements DTORG as one of the nonfunctional directives supported for source-level compatibility reasons.

- f) A P?S/8 utility, tentatively to be known as BINCON, may be written to read and convert present OS/8 binary files to the proposed format [with respect to the DTORG disposition]; the resulting compatible binary format would then be stored as P?S/8 *slurp* binary format files. This would implement binary compatibility with the proposed changes to DTORG as implemented in future releases of P?S/8 PAL. P?S/8 system generation utilities could operate with binary paper tapes or other media developed on either operating system.

Note: P?S/8 BIN can create binary paper tapes in BIN format on the low-speed or high-speed punch; this can be accomplished by a chained assemble-and-punch operation as described elsewhere in this document.

The reader is directed to various documents available on the Internet covering the fundamentals of PDP-8 programming and the PAL language as implemented on various systems [not all of which are compatible with DEC standards]. P?S/8 PAL is generally a superset of all other compatible implementations [to within nitpicks].

Additional information on the P?S/8 PAL assembler is available on the Internet at:

<http://www.ibiblio.org/pub/academic/computer-science/history/pdp-8/§/PQS8-related%20Files/1989%20Help%20Files/ASMBLR.HELP>

Note: The § above indicates continuation of the URL address on the next line; the address was split into two lines due to overall length.

A short summary of all supported command line option switches is contained within this file along with other internal documentation of P?S/8 PAL taken from the source code file [ASMBLR.PAL].

FLIP.EXE-related issues.

Proper usage of the online copy of ASMBLR.HELP may require end-of-line conversion due to unix conventions used on the server.

All text files [including the aforementioned ASMBLR.HELP] located within the archive referenced above can be converted to MS-DOS/Windows conventions using Rahul Dhesi's popular FLIP.EXE program.

FLIP can convert files between MS-DOS and unix conventions in both conversion directions. Running the program without arguments from an MS-DOS window will output the program options needed for a successful conversion.

FLIP.EXE is widely available as freeware; a copy of this program is included in the P?S/8 Advanced SIMH Package [PASP] in the \PDP-8\SIMH\PDP-8\SIMH Utilities directory.

Note: Due to limitations of FLIP.EXE, it may be necessary to use file names conforming to MS-DOS conventions; many files associated with the P?S/8 project use naming conventions compatible with the forthcoming P?S/8 SHELL directory structure. [The P?S/8 SHELL uses 12.4 file names with case retention while MS-DOS uses 8.3 file names in upper-case only]. Other than file date limitations, SHELL directory files are easily stored in windows directories; files may be renamed to MS-DOS conventions in windows for further processing with FLIP.EXE, then renamed back to the original file name as required.

Appendix A.

P?S/8 PAL Error Messages.

Error	Meaning
BE	Current page literal Buffer Exceeded. Only 63 current page literals are allowed per page.
BO	Binary output Overflow. Insufficient binary output files passed to complete the assembly.
CO	Conditional assembly syntax error or nested < or > count problem.
DT	Duplicate Tag error. An attempt was made to modify an existing symbol value using ,.
ER	User generated Error. The program includes internal logical safeguards that were violated. See Note 1.
FI	Fixmri error. An attempt was made to improperly define an MRI type symbol or redefine a permanent symbol.
IC	Illegal Character error. Possibly [or (usage without enabling literals.
IE	Illegal Expression error. The latest statement is syntactically malformed.
II	Illegal Indirect error. The designated pointer is not located on the current page or page zero.
IM	Insufficient Memory to load the symbol table. This may be remedied by using the /C option switch.
IP	Illegal Pseudo-op [directive] redefinition attempt. All directives are permanent symbols and cannot be redefined.
IR	Illegal Reference to an inaddressable memory location by a direct reference instruction [instead of a pointer address].
LG	Unsuppressed Link Generated error with link generation enabled. The generated link will also be flagged.
NE	Null Expression within a literal reference. The expression must contain at least one term.
NU	Numerical data error. This is usually an attempt to use 8 or 9 when the prevailing radix is octal.

Appendix A [continued].

PA	PA use message. A programmed value associated with this message is displayed. See Note 2.
PE	P age E xceeded. The code on a current overlaps the literal table. The literal table [or program] must be made smaller.
PH	Conditional assembly P Hase error. The assembly terminated with an open failing conditional assembly statement.
PO	P ushdown list O verflow. The latest expression is too complex; simplify the code using a series of smaller steps.
ST	S ymbol T able exceeded. More memory is required [or use less modular assembly options].
SY	Miscellaneous S Yntax error. The error doesn't fall into any of the other reported categories.
UF	U ndefined F ield error. The argument to a FIELD directive is undefined.
UO	U ndefined O rigin error. The argument to an origin setting using * is undefined.
US	U ndefined S ymbol error. A symbol was referenced that was never defined by the end of pass one of the assembly.
ZE	Page Z ero E xceeded. Page zero code overlaps the literal table; the program or literal table must be made smaller.

Notes.

- 1) well-written programs include internal safeguards to prevent scenarios that violate obscure nuanced details of the program such as requiring a particular symbol to be maintained within a narrow address range [or at a particular location in memory]. Analysis of the error message report may help the maintenance programmer determine what programming blunder was committed.
- 2) The PAuse message is not an error. Well- developed programs occasionally include reports during the assembly for a variety of purposes, including providing additional information about an error condition signaled by the ERROR directive. Extremely long source files may include the PAUSE directive to report when an assembly pass starts [or finishes].

End of Appendix A.

Appendix B.

P?S/8 PAL Directives.

ASMIFM *expression*

The ASMIFM directive is one of the conditional assembly features of the extended PAL language. See Note 1.

The specified *expression* is evaluated. If the result is negative, the next line is assembled. If the result is positive, the next line is ignored.

```
ASMIFM      A-B                /COMPARE A TO B.  
D=3          /SET D TO 0003.
```

The next line may be blank or consist of comments only rendering the conditional moot. During the early development of a program, this may deliberately be done to force a specific outcome.

Note: If additional statements are on the same line as the ASMIFM directive [using ; to separate the statements] and the conditional test fails, the additional statements are also ignored.

ASMIFN *expression*

The ASMIFN directive is one of the conditional assembly features of the extended PAL language. See Note 1.

The specified *expression* is evaluated. If the result is non-zero, the next line is assembled. If the result is zero, the next line is ignored.

```
ASMIFN      A-B                /COMPARE A TO B.  
D=3          /SET D TO 0003.
```

The next line may be blank or consist of comments only rendering the conditional moot. During the early development of a program, this may deliberately be done to force a specific outcome.

Note: If additional statements are on the same line as the ASMIFN directive [using ; to separate the statements] and the conditional test fails, the additional statements are also ignored.

Appendix B [continued].

ASMIFZ *expression*

The ASMIFZ directive is one of the conditional assembly features of the extended PAL language. See Note 1.

The specified *expression* is evaluated. If the result is zero, the next line is assembled. If the result is non-zero, the next line is ignored.

```
ASMIFZ    A-B                /COMPARE A TO B.  
D=3       /SET D TO 0003.
```

The next line may be blank or consist of comments only rendering the conditional moot. During the early development of a program, this may deliberately be done to force a specific outcome.

Note: If additional statements are on the same line as the ASMIFZ directive [using ; to separate the statements] and the conditional test fails, the additional statements are also ignored.

ASMSKP *expression*

The ASMSKP directive is one of the conditional assembly features of the extended PAL language. See Note 1.

The specified *expression* is evaluated as an unsigned 12-bit integer value. The next *expression* value lines of the source code are ignored.

```
ASMSKP    3                  /SKIP THE NEXT THREE LINES.  
HLT       /THIS IS IGNORED!  
JMP       .+3                /THIS IS ALSO IGNORED!  
/                          IGNORED AND ALSO A COMMENT!
```

If the *expression* evaluates to zero, all lines following the ASMSKP directive will be assembled. If the *expression* evaluates to a non-zero value and additional statements are on the same line as the ASMSKP directive [using ; to separate the statements], the additional statements are also ignored.

Note: The need for the ASMSKP directive comes about because of the complex nature of conditional assembly using the provided directives [certain logical expressions might require the use of multiple conditionals]. Unfortunately, since the ASMSKP directive does not work with any form of delimiter mechanism, the programmer must carefully count the lines expected to be skipped; changes to the program source can cause unexpected results.

Appendix B [continued].

CONSOL{E}

The front panel switches are read during the assembly.

The CONSOL{E} directive returns the contents of the front panel switches to allow storage of the value into an assembly-time symbol:

```
SW=  CONSOL          /STORE CONSOLE SWITCHES IN SW.
```

Conditional assembly can use the front panel switch register value for a variety of purposes as determined by the programmer.

DATE

The DATE directive returns the low-order 12-bits of the current system date to allow storage of the value into an assembly-time symbol:

```
CURDAY=  DATE          /STORE THE DATE IN CURDAY.
```

Conditional assembly can use the date for a variety of purposes including creating a current version of the program.

Note: Future releases of P?S/8 PAL will likely include the DATEHI directive to obtain the high-order six bits of the full current system date. The two values can be combined to form the complete date [in the range of 01-JAN-1900 through 31-DEC-2411].

DECIMA{L}

The internal radix for mathematical expressions is set to DECIMA{L}; the default radix is OCTAL.

DEVICE

The DEVICE directive takes a four-character string argument. It is used to form the device-name string used to reference P?S/8 non-system device handlers and OS/8 device handlers:

```
DEVICE  DTA1          /REFERENCE TO DTA1:
```

The DEVICE directive creates two 12-bit words containing the packed six-bit ASCII text characters of the argument. In the example given, the output would consist of 0424 followed by 0161.

Note: The DEVICE directive is not affected by the setting of the /J option switch.

Appendix B [continued].

DTORG *expression*

The DTORG directive is intended to provide target storage block number information when generating operating system components in systems such as Typeset-8. See Notes 1, 2 and 3.

The *expression* is evaluated as a 12-bit unsigned integer to designate a target storage block on a device such as TC01/TC08/TU55/TU56 DECTape.

Note: The implementation of the DTORG is extremely problematic; see section 12 of this document for more information.

EJECT *optional-string-value*

The EJECT directive causes assembler listing output to advance to the next page.

If used with the *optional-string-value*, the title on the next page is set to the *optional-string-value* text [subject to truncation].

Note: Statements containing the EJECT directive are not shown in the assembler listing output.

ENBITS

The ENBITS directive is used in TOPS-10 PAL10 to enable the bitmap of the latest memory field when either a new FIELD directive is used or the end of the assembly. This feature is not implemented in any other PAL assembler; the ENBITS directive is provided for compatibility purposes only. See Note 2.

ENDBIN

The ENDBIN directive closes the current binary file. Additional binary output [if any] will be stored in the next binary output file.

The ENDBIN directive is ignored if binary output generation is not enabled in the overall assembly.

The ENDBIN directive is useful when creating optional program segments from a master assembly file to be loaded separately. It is supported primarily for compatibility with the paper-tape system assembler PAL III.

Appendix B [continued].

ENPUNC{H}

Binary output generation may have been turned off; the ENPUNC{H} directive [re-]enables binary output generation.

The ENPUNC{H} directive is ignored if binary output generation is not enabled in the overall assembly. See Note 4.

ERROR *optional-numeric-expression*

The ERROR directive causes a deliberate error and error message which will prevent automatic chaining to P?S/8 BIN.

Well developed PDP-8 programs may contain safeguards to prevent undesirable scenarios such as having a definition fall out of a required narrow range of addresses [or in some cases a unique required value]. This will help future program maintainers from making logical blunders.

The value of the *optional-numeric-expression* will be displayed as part of the error message to help pinpoint the problem:

*7

```
LOCTM1,    .-.                /TEMPORARY STORAGE.
LOCTM2,    .-.                /ANOTHER TEMPORARY [WRONG PLACE!]

AUTOXR=     .                /THIS IS WRONG! SHOULD BE 0010.

MYPTR=     AUTOXR+7          /USED AS AN AUTO-INDEX REGISTER.

IFNZRO     MYPTR&7770-10    <ERROR MYPTR>
```

The value of MYPTR is 0020 instead of staying within the required range of 0010 through 0017; as such, it is not capable of being used as an auto-index register as the program logic demands. Hopefully, the error message `ER 0020 AT LOCTM2+0001 (0011)` [where 0011 is the current location counter of the assembly in this example], is sufficient to aid the maintenance programmer in correcting the mistake. [The LOCTM2 location is inappropriately placed in the auto-index register area ruining the program logic. By moving LOCTM2 to another area of page-zero memory, the problem is eliminated; program reassembly will not trigger the error-detection safeguard.]

Note: If the *optional-numeric-expression* is not used, the error message will output `ER 0000`.

Appendix B [continued].

EXPUNG{E}

The EXPUNG{E} directive is used to clear the entire symbol table at the start of an assembly.

All permanent symbols are removed [except directives]. The program must define all symbols used [including MRI symbols using the FIXMRI directive].

By deleting the internal symbol table, it is possible certain programs may be capable of assembly on small memory configurations.

Note: The source code of P?S/8 PAL performs an EXPUNG{E} at the beginning of the assembly; P?S/8 PAL supports many symbols not ordinarily found in many assemblers [such as IOT instructions for several peripherals and PDP-8/E EAE].

FIELD *optional-numeric-expression*

The FIELD directive is used to set a designated memory field for binary loading of statements that follow. If the *optional-numeric-expression* is present, it is evaluated to set the desired field; if there is no argument, the next memory field is selected.

When a FIELD directive is used, all open literal pools are output and reset to default values. If the /0 option switch is set, an automatic *200 will also be generated overriding the previous value of the current location counter.

Note: Memory fields up to a maximum of 256K are supported for compatibility with the KT8A option; however, all standard binary loaders can only load binary generated in the normal 32K address space [field 0 through field 7]. Future releases of P?S/8 may include a KT8A-specific loader that will require the optional hardware to operate.

FILENA{ME}

The FILENA{ME} directive is used to create a four word text string consisting of an OS/8 six-character file name and a two-character file extension:

```
FILENAME  FOOBAR.PA /STATIC FILE NAME STRING.
```

The FILENA{ME} directive creates four 12-bit words containing the packed six-bit ASCII text characters of the argument. In the example given, the output would consist of 0617, 1702, 0122 and 2001.

Appendix B [continued].

P?S/8 supports the FILENA{ME} directive for compatibility with OS/8 PAL8. Future P?S/8 systems will support the P?s/8 SHELL with an extended assembler based on P?S/8 PAL to be known as XPAL. XPAL will support an option to extend the FILENA{ME} directive to support SHELL directory file names.

Note: The FILENA{ME} directive is not affected by the setting of the /J option switch.

FIXMRI

The FIXMRI directive is used to modify a standard equate statement so the assembler treats the symbol as a Memory Reference Instruction [MRI]. MRI symbols modify the expression evaluation so that the PDP-8 addressing rules are applied instead of simple inclusive OR [.IOR] logic which is the default method of expression evaluation:

```
FIXMRI      INC= 2000          /ISZ PRESUMED TO NEVER SKIP.
```

The FIXMRI directive is also used to define floating-point pseudo-instructions that generally follow the MRI addressing rules as part of several software floating-point packages.

When the EXPUNG{E} directive is used, FIXMRI is needed to redefine the standard symbols that are MRI type [such as TAD, DCA, etc.].

FIXTAB

The FIXTAB directive is used after certain symbols are defined that the programmer desires to be treated as permanent symbols. All such initialization should occur at the start of the assembly to establish the permanent symbol table [perhaps after an EXPUNG{E} directive].

All permanent symbols will not be included in the cross-reference output [if the /X option switch is in effect] or the symbol table printout at the end of the assembly [if the /S option switch is in effect] unless the /A option switch is also set.

I

The I directive is used in all PDP-8 MRI statements involving indirect references. This forces bit[3] to be set so the generated instruction conforms to the PDP-8 addressing rules [after the evaluated expression is adjusted according to the MRI addressing rules in effect] for indirect addressing. See Note 5.

Appendix B [continued].

IFDEF *symbol-name* <*statements*>

The IFDEF directive is one of the conditional assembly features of the PAL language.

The symbol table is searched for the specified *symbol-name*. If the symbol is present and defined, the *statements* contained within the < and > are assembled. If the symbol is either not present or not defined at that point in the assembly, the *statements* contained within the < and > are ignored.

The *statements* can span an indefinite number of lines of source code as necessary. Conditionals are occasionally used within other conditionals in a "nested" manner. The count of < and > must be carefully maintained to achieve an error-free assembly.

Note: The contents of comments are scanned for < and > when conditional assembly is in effect.

The IFDEF directive is often used to include other symbols or data into an assembly if relevant:

```
IFDEF      DECTAPE    <WC= 7754>      /DEFINE WORD COUNT.
```

Note: Unlike ordinary references to symbols, attempting to locate an undefined symbol does not cause an error.

IFDEF *symbol-name* <*statements*>

The IFNDEF directive is one of the conditional assembly features of the PAL language.

The symbol table is searched for the specified *symbol-name*. If the symbol not present, the *statements* contained within the < and > are assembled. If the symbol is present and defined at that point in the assembly, the *statements* contained within the < and > are ignored.

The *statements* can span an indefinite number of lines of source code as necessary. Conditionals are occasionally used within other conditionals in a "nested" manner. The count of < and > must be carefully maintained to achieve an error-free assembly.

Note: The contents of comments are scanned for < and > when conditional assembly is in effect.

Appendix B [continued].

The IFNDEF directive is often used to initialize a symbol default value:

```
IFNDEF    NEWDATA    <NEWDATA=1>    /GET NEW DATA.
```

In the example above, the conditional statement is assembled during pass one of the assembly. During pass two the statement is ignored because the symbol is defined.

Note: Unlike ordinary references to symbols, attempting to locate an undefined symbol does not cause an error.

IFNZRO *expression* <*statements*>

The IFNZRO directive is one of the conditional assembly features of the PAL language.

The specified *expression* is evaluated. If the result is non-zero, the *statements* contained within the < and > are assembled. If the result is zero, the *statements* contained within the < and > are ignored.

The *statements* can span an indefinite number of lines of source code as necessary. Conditionals are occasionally used within other conditionals in a "nested" manner. The count of < and > must be carefully maintained to achieve an error-free assembly.

Note: The contents of comments are scanned for < and > when conditional assembly is in effect.

The IFNZRO directive is often used to check an assembly parameter range:

```
AUTOXR=    11                /THIS IS WRONG!  SHOULD BE 0010.  
MYPTR=     AUTOXR+7          /USED AS AN AUTO-INDEX REGISTER.  
IFNZRO     MYPTR&7770-10    <ERROR MYPTR>
```

In the example above, the range of MYPTR must be between 0010 and 0017. If this is not correct, an error message is issued.

Appendix B [continued].

IFZERO *expression* `<statements>`

The IFZERO directive is one of the conditional assembly features of the PAL language.

The specified *expression* is evaluated. If the result is zero, the *statements* contained within the `<` and `>` are assembled. If the result is non-zero, the *statements* contained within the `<` and `>` are ignored.

The *statements* can span an indefinite number of lines of source code as necessary. Conditionals are occasionally used within other conditionals in a "nested" manner. The count of `<` and `>` must be carefully maintained to achieve an error-free assembly.

Note: The contents of comments are scanned for `<` and `>` when conditional assembly is in effect.

The IFZERO directive is often used to create assembler default values:

```
IFZERO    OURFLD    <
BUFFER=    4000                /USE FIELD 0 VALUE.>
IFZERO    OURFLD-1 <
BUFFER=    6000                /USE FIELD 1 VALUE.>
```

In the example above, a parameter must be set to a value consistent with an overall assembly parameter that can have multiple values.

LMODE

The LMODE directive is provided in the dual mode assembly module to enable LINC mode addressing and certain symbols unique to LINC mode. See Note 1.

NOBITS

The NOBITS directive is used in TOPS-10 PAL10 to disable the bitmap of the latest memory field when either a new FIELD directive is used or the end of the assembly. This feature is not implemented in any other PAL assembler; the NOBITS directive is provided for compatibility purposes only. See Note 2.

Appendix B [continued].

NOPUNC{H}

The NOPUNC{H} directive [temporarily] turns off binary output generation during the assembly without affecting the assembler's current location counter.

The NOPUNC{H} directive is ignored if binary output generation is not enabled in the overall assembly. See Note 4.

OCTAL

The internal radix for mathematical expressions is set to OCTAL. While the default radix is OCTAL, an earlier section of the source file may have changed the radix to DECIMA{L}. Any programming section where the current radix is in doubt should include a usage of the OCTAL directive.

PAGE *optional-numeric-expression*

The PAGE directive is used to set an origin setting at the lowest location on a PDP-8 memory page for binary loading of statements that follow. If the *optional-numeric-expression* is present, it is evaluated to set the desired page; if there is no argument, the next memory page is selected.

Selecting a new page will cause the output of the literal table of the previous page in effect.

Note: The default origin of any assembly is 0200; this is equivalent to using the PAGE 1 directive.

PAUSE *optional-numeric-expression*

The PAUSE directive has two different usages:

When used without the *optional-numeric-expression*, the PAUSE directive indicates the rest of the latest input file will be ignored. This usage of the PAUSE directive is provided for compatibility with the paper-tape operating system PAL III assembler.

When used with the *optional-numeric-expression*, a PAUSE message is displayed during each pass of the assembly. This is a user-defined function and is not an error message per se [although the programmer can define values to specific messages to describe some internal situation].

When developing extremely long programs, programmers may use the PAUSE directive to signal the program section being processed and/or the start [or end] of each assembly pass. For more information see Section 3 of this document.

Appendix B [continued].

PMODE

The PMODE directive is provided in the dual mode assembly module to enable PDP-8 mode addressing and certain symbols unique to PDP-8 mode. See Note 1.

PQS

The PQS directive is a unique feature of P?S/8 PAL. It can be used in conditional assembly statements to allow special-purpose programming only supported by P?S/8 PAL; alternate statements can attempt to provide compatibility with other assemblers. Unfortunately, certain powerful features of P?S/8 PAL are not available in other assemblers; crude alternatives have been used to provide a functional, albeit inferior, result.

Features of P?S/8 PAL such as conditional literals have been instrumental in implementing several very large programs; conditional assembly has been used to provide inferior support of statically-determined page zero literals through the use of manual optimization at some point in the development. However, as additional development is performed, the static manual optimization tends to drift while assembly with P?S/8 is always current. Only additional manual optimization can sync up the relevant changes. As such, the PQS directive has been used extensively to allow OS/8 PAL8 to replicate the latest static optimization until further development with P?S/8 can be performed.

Note: The forthcoming P?S/8 SHELL will include a more advanced assembler partially based on P?S/8 PAL tentatively to be known as XPAL. P?S/8 SHELL XPAL will also support the PQS directive; however, since XPAL will be a more comprehensive assembler, a companion PQSX directive will be implemented. This will allow programmers to take advantage of the advanced features of both assemblers.

RELOC *optional-numeric-expression*

The RELOC directive performs much of the functionality of the NOPUNC{H} and ENPUNC{H} directives in a more compact and elegant form. The *optional-numeric-expression* is evaluated to form a relocation factor that sets up an offset between the assembler's current location counter and the binary output origin.

Since the relocation factor is applied to all binary output, the RELOC directive can be used with literals.

Appendix B [continued].

Additional usage of the RELOC directive with the *optional-numeric-expression* causes cumulative relocation, which can be useful in certain obscure situations. The RELOC directive used without the *optional-numeric-expression* resets the relocation factor to zero.

Note: While OS/8 PAL8 is the only other assembler that supports the RELOC directive, there are certain differences:

- a) OS/8 PAL8 flags statements detected as *apparently* relocated with *. P?S/8 PAL flags statements in a similar manner, but only if relocation is actually in effect.

If a statement of the form

RELOC .

is used in the source program, relocation is reset to zero; however, OS/8 PAL8 will only cancel the statement relocation flag when the RELOC directive is used without the *optional-numeric-expression*.

- b) OS/8 PAL8 does not flag statements that are relocated by the use of the NOPUNC{H} and ENPUNC{H} directives.

P?S/8 PAL outputs the * on all statements actually relocated regardless of method of relocation.

Note: Certain advanced programming methods include the simultaneous usage of the NOPUNC{H} and ENPUNC{H} directives and the RELOC directive; this is necessary when there are several independent aspects of relocation in affect at the same time.

SEGMNT *optional-numeric-expression*

The SEGMNT directive is used to set the field and origin setting to the quarter memory field segment specified by the *optional-numeric-expression* in the range of 00 [field 0, location 0000] through 37 [field 7, location 6000] as required by the LINC architecture of the LINC-8 and PDP-12.

If the *optional-numeric-expression* is present, it is evaluated to set the desired segment; if there is no argument, the next memory segment is selected

Note: Segment 2 is the default value for LINC mode assembly followed by an origin of 0020 (04020). This is equivalent to SEGMNT 2 followed by *20 [in LINC mode, addresses are specified within a segment]. See Note 1.

Appendix B [continued].

SIXBIT *text-string*

The SIXBIT directive creates 12-bit words packed with six-bit characters taken from the *text-string* [other than delimiter characters]. The encoding is six-bit ASCII text as used with similar directives in assemblers for DEC 18-bit and 36-bit systems [which is not the PDP-8 standard].

Note: If the number of characters in the *text-string* is even, an additional 12-bit word of 0000 is created as a delimiter unless the /J option switch is set. For more information see Section 10 of this document.

SKIP *optional-numeric-expression*

The SKIP directive is used to create deliberately blank lines in the listing output of an assembly. The *optional-numeric-expression* is evaluated as an unsigned 12-bit integer. Blank lines are created as many times as necessary unless the value is zero, in which case no blank lines are created. If the *optional-numeric-expression* is not present, one blank line will be created.

Note: All lines containing the SKIP directive will not be displayed in the listing.

TEXT *text-string*

The TEXT directive creates 12-bit words packed with six-bit characters taken from the *text-string* [other than delimiter characters]. The encoding is six-bit ASCII text as typically used on the PDP-8 [which is not compatible with similar directives in assemblers for DEC 18-bit and 36-bit systems].

Note: If the number of characters in the *text-string* is even, an additional 12-bit word of 0000 is created as a delimiter unless the /J option switch is set. For more information see Section 10 of this document.

TITLE *text-string*

The TITLE directive is used to set the title field of the page header used on every listing page if the /L and /N option switches are set. The *text-string* will be used [subject to truncation] starting with the next printed page.

Appendix B [continued].

Normally, the title field is taken from the first line of the first input file; the TITLE directive can only affect subsequent output. However, the TITLE directive can be used to override the first page title field by using the XLIST directive beforehand:

```
/      EXAMPLE WHERE THIS IS NOT USED IN THE TITLE FIELD.

XLIST                                /TURN OFF LISTING NOW.

TITLE      MY TITLE                  /USE DESIRED TITLE INSTEAD.

XLIST                                /TURN LISTING ON NOW.
```

Note: All lines containing the XLIST directive are hidden.

XLIST *optional-numeric-expression*

The XLIST directive is used to hide portions of a listing file [assuming the /L option switch is set]. If the *optional-numeric-expression* evaluates to a non-zero value, the listing is turned off. If it evaluates to zero, the listing output is restored. If no argument is given, the listing output state is reversed from its former state.

Note: All lines containing the XLIST directive are hidden.

Z

The Z directive is provided for compatibility with the PAL II assembler of the paper-tape operating system. All PDP-8 assemblers starting with PAL III are capable of properly setting arguments to MRI type instructions according to the rules of the PDP-8 architecture. PAL II only implements a subset where all symbols are erroneously placed on the current page. When necessary, the Z directive is used on all statements that actually address page zero to remove the current page bit from the generated value.

Note: PAL III was originally written in PAL II. Later releases of the source code were modified to require a binary copy of PAL III to assemble its own source code.

ZBLOCK *numeric-expression*

The ZBLOCK directive is used to create blocks of data set to 0000. The *numeric-expression* is evaluated as an unsigned integer; the value determines how many words are generated.

Note: If the *numeric-expression* evaluates to zero, no words are generated.

Appendix B Notes.

- 1) Certain directives are only available if LINC mode dual assembly is enabled using either the /8 or /9 option switches. They are provided primarily for compatibility with the DIAL-MS dual mode assembler. Due to the nature of LINC mode expressions that use one's complement arithmetic, the programmer must be aware of the attendant quirks [such as positive and negative zero values].
- 2) Certain directives are erroneously provided only if dual mode assembly is enabled using either the /8 or /9 option switches. These directives will be moved to the general symbol table load in a future release; for the present, dual mode assembly must be enabled.
- 3) The DTORG directive should not be used at this time as its presence in the source code causes the binary output to be corrupted. See Section 12 of this document for more information.
- 4) The NOPUNC{H} and ENPUNC{H} directives are often used to relocate binary output as part of an overlay structure. The assembler current location counter can be updated while the binary output origin remains fixed. The program segment loads into an address space that can be moved or stored elsewhere while properly assembled for its ultimate execution address space:

*6000	/WHERE THE GENERATED CODE LOADS.
NOPUNCH	/FOOL THE ASSEMBLER.
*7600	/WHERE THE CODE EXECUTES.
ENPUNCH	/UN-FOOL THE ASSEMBLER.

Note: The NOPUNC{H} and ENPUNC{H} directives are traditional features of several PDP-8 assemblers including TOPS-10 PAL10. Literals cannot be used with this form of relocation mechanism.

The RELOC directive can accomplish a similar form of code relocation including literals if necessary. However, it is only supported by P?S/8 PAL and OS/8 PAL8.

Appendix B Notes [continued].

- 5) The I directive changes meaning if LINC mode assembly is in effect. Since there are no indirect references in LINC programming, the I directive is redefined to force bit[7] on. In the context of LINC addressing, the I directive means either immediate mode [when used with instructions where the operand immediately follows the instruction] or auto-indexed mode [when used with references to locations 0001-0017].

Many LINC instructions take immediate arguments:

```
LMODE                      /LINC MODE ASSEMBLY.

LDA I;    3000             /LOAD THE AC WITH 3000.
ADA ;     FOO              /ADD THE CONTENTS OF FOO
```

The first instruction above uses the I bit to indicate the argument [3000] is immediately after the instruction. The second instruction does not use the I bit; as such, the address of the operand follows [which ironically means it is an indirect reference to the actual operand].

The PDP-8 references to locations 0010-0017 are auto-incremented when referenced indirectly. In LINC mode, references to 0001-0017 are ordinary indirect references unless the I directive is used:

```
LMODE                      /LINC MODE ASSEMBLY.

ADA    17                 /ADD WHAT IS POINTED TO.
ADA I  17                 /ADD NEXT VALUE AS WELL.
```

The first instruction uses the present contents of 0017 to point to the operand. The second instruction first increments the contents of 0017 to point to the [next] operand.

Certain LINC instructions have special meaning for references to location 0000; these instructions have specific addressing needs which do not involve the I bit. The auto-increment references to locations 0001-0017 clearly are the inspiration for the PDP-8 auto-increment usage of locations 0010-0017. The range was shortened to allow PDP-8 interrupt handling at the lower addresses. [The LINC was designed without an interrupt structure; however, some unofficial add-ons were known to exist. The LINC-8 does not support LINC interrupts. The PDP-12 defines a LINC mode interrupt analogous to the PDP-8 interrupt but to an alternate location.]

Appendix B Notes [continued].

The PDP-8 cannot use references to 0010-0017 without auto-increment; to this limited extent, the LINC instruction set is superior to the PDP-8 instruction set. LINC mode addressing does not allow indirect usage of any other locations [other than pointer locations immediately after the instructions]. There is no concept of a page zero in the LINC architecture, and there are only three instructions that address all of memory directly. However, all of memory in this case is limited to 1024 locations, not the PDP-8 norm of 4096 locations. There is no concept of a subroutine call mechanism; the JMP instruction attempts to act as a stand-in. However, if JMP instructions are used in the subroutine, various fixups [read kludges here] are needed to protect the return address. All features considered, system architecture experts are welcome to way in with their own opinions which may greatly differ with the author's viewpoint.

End of Appendix B.

[End-of-file]